

PathBlocker AI

Project Proposal

An AI Powered Strategic Board Game

Adversarial search and heuristic evaluation applied to an interactive, real time web game.

1. Introduction

PathBlocker AI is a two player strategy board game, inspired by the classic abstract game Quoridor, reimagined with a modern neon styled interface and a computer controlled opponent powered by artificial intelligence. Players race to move their pawn across a 9x9 grid to the opposite edge of the board while strategically placing walls to obstruct their opponent's shortest path. The project combines classical game tree search algorithms with a full stack web application, allowing a human player to compete against an AI bot of adjustable difficulty, or against another human locally.

The purpose of this project is to demonstrate the practical application of adversarial search and heuristic evaluation, core topics in Artificial Intelligence, within an interactive, real time, user facing product, rather than a purely theoretical or command line exercise.

2. Problem Statement

Turn based strategy games with imperfect but bounded state spaces (such as Quoridor, Chess, and Checkers) present a well known challenge in AI: how to build an opponent that plays competently, responds within a reasonable time, and remains enjoyable to play against, neither trivially weak nor unbeatably strong.

Specifically, this project addresses:

- **Decision making under a large branching factor:** at any turn a player may move their pawn or place one of up to 128 possible walls, making brute force search computationally expensive.
- **Balancing AI strength with responsiveness:** a naive full depth search would be too slow for real time play; the AI must return a move within a few seconds without feeling unresponsive.
- **Adjustable difficulty:** different users have different skill levels, so the AI needs multiple tiers of challenge rather than a single fixed strategy.
- **Delivering this as an accessible, engaging product:** the underlying algorithm is only useful if wrapped in an interface people actually want to play with.

3. Software / Tools Used

Category	Tool / Technology	Purpose
Backend language	Python 3	Core application and AI logic
Web framework	Django	Serves the game UI and exposes the AI as an API endpoint
Frontend	HTML5, CSS3, JavaScript (Canvas API)	Renders the interactive game board and handles user input
AI / Algorithms	Custom Python implementation	Minimax search, alpha-beta pruning, BFS pathfinding
Data interchange	JSON over HTTP (Fetch API)	Communicates board state between browser and server

Version control	Git	Source control and change tracking
Development environment	Django development server	Local testing and iteration

4. Methodology / Algorithm

The AI opponent is built around **minimax search with alpha-beta pruning**, a standard adversarial search algorithm from game theory and AI:

1. **State representation:** the board is modeled as pawn positions, wall placement grids (horizontal/vertical), and remaining wall counts per player.
2. **Move generation:** legal pawn moves are generated per Quoridor's movement rules (including the jump over an adjacent opponent special case); legal wall placements are generated and filtered to exclude any wall that would completely block a player's path to their goal (verified via reachability check).
3. **Search:** the algorithm recursively simulates future turns up to a fixed depth, alternating between maximizing the AI's outcome and minimizing the opponent's. Alpha-beta pruning eliminates branches that cannot influence the final decision, substantially reducing computation.
4. **Heuristic evaluation function:** since the tree cannot be searched to completion in real time, each leaf node is scored using Breadth First Search (BFS) shortest path distance for each player to their target row as the primary component, plus remaining wall count and pawn mobility.
5. **Move ordering:** candidate moves are pre-sorted by a shallow one ply evaluation before deep search, so the strongest looking moves are explored first, improving the effectiveness of alpha-beta pruning.
6. **Difficulty tiers:** Easy uses greedy shortest path movement with occasional wall placement; Medium runs full minimax search to depth 2; Hard runs full minimax search to depth 3 with a bounded time budget.
7. **Time boxing:** a deadline safeguard ensures the AI always returns a move within its time budget, falling back to the best move found so far if search is interrupted.

5. Expected UI Elements

- **Main menu screen:** mode selection (vs. AI / 2 Player local), AI difficulty selector, and a visual theme picker.
- **Game board:** a 9x9 grid rendered on an HTML5 canvas, with clear visual distinction between cells, placed walls, and pawns.
- **Player HUD:** live indicators for each player's turn, remaining wall count, and an "AI is thinking" status during bot computation.
- **Action controls:** buttons and keyboard shortcuts to switch between Move, Place Horizontal Wall, and Place Vertical Wall modes.

- **Move/placement hover previews:** highlighting valid moves and wall positions before commitment, with invalid actions visually disabled.
- **Game over screen:** announces the winner with a restart/play again option.
- **Theme system:** multiple selectable visual styles (Neon, Glassmorphism, Cyberpunk, Synthwave, Minimal) to personalize the experience.

6. Expected Outputs / Inputs

Inputs

- Player selections from the menu: game mode, AI difficulty, visual theme.
- In-game actions: mouse/touch clicks or taps to move a pawn or place a wall; keyboard shortcuts for switching action modes.
- Board state sent to the backend (pawn positions, wall layout, remaining walls, current turn) whenever it is the AI's turn.

Outputs

- Rendered game board reflecting the current state after every move.
- The AI's chosen action (pawn move or wall placement), computed server side and returned as a JSON response.
- Real time UI feedback: turn indicators, wall counts, invalid action messages, and win/loss announcements.
- A persisted theme preference (stored locally) applied across sessions.

7. Conclusion

PathBlocker AI demonstrates how a foundational AI search technique, minimax with alpha-beta pruning, guided by a BFS based heuristic, can be applied to build a competent, adjustable, and responsive game playing agent, and how that agent can be delivered through a polished, real time web interface. The project bridges theoretical AI coursework with practical software engineering: state modeling, search algorithms, heuristic design, API design, and frontend interactivity. Beyond its educational value, the result is a genuinely playable strategy game that can serve as a portfolio piece, a base for further AI experimentation (e.g., deeper search, learned evaluation functions, or reinforcement learning agents), and a demonstration of full stack development skills using Python and Django.